Benchmarking GGUF Quantization Variants of Llama 3.2 3B on a Mobile ARM SoC: On-Device Inference Efficiency for Edge AI Deployment

Kris D'Costa
Halicioglu Data Science Institute
University of California, San Diego
San Diego, CA, USA
kdcosta@ucsd.edu

***Abstract*—We present a systematic empirical study of six GGUF quantization variants of Llama 3.2 3B Instruct on a Google Pixel 6a (Tensor G1, 6 GB LPDDR5) using llama.cpp, yielding 661 valid measurements across five quantization levels, four context lengths (256–2048 tokens), and a thread-count sweep. We find four non-obvious results. (1) Throughput is non-monotonic with bit-width: Q2_K leads at 5.66 tok/s while Q6_K is *slowest* at 3.98 tok/s, 20% below Q8_0, due to kernel dequantization overhead on ARM NEON. (2) Context-length sensitivity is variant-dependent: Q4_K_M ($+6\%$) and Q2_K (-4%) are stable to 2048 tokens; Q8_0 declines mildly (-11%); while Q3_K_M drops 38% and Q6_K drops 49% at ctx=2048, exposing a hidden memory-system fragility in these two quantization schemes. (3) Accuracy is also non-monotonic: on a 100-question BoolQ reading comprehension benchmark, Q4_K_M (72%, 95% CI: 63–80%) achieves the highest accuracy while Q6_K scores lowest (65%, CI: 55–74%), despite having more bits per weight. On 100-question ARC-Easy factual retrieval, all quantized variants score 95–100% (near-ceiling), confirming the BoolQ result as the primary quality discriminator. (4) Q6_K is strictly dominated: simultaneously the least accurate and the slowest at long context; the Pareto frontier is defined by Q4_K_M (accuracy-dominant, 72%, 5.32 tok/s) and Q2_K (speed-dominant, 69%, 5.66 tok/s). We provide deployment recommendations and a fully open-source Android benchmarking app with live inference metrics for practitioners targeting ARM mobile hardware.**

***Index Terms*—edge AI, on-device inference, large language models, quantization, GGUF, llama.cpp, mobile SoC, ARM, Android, Llama 3.2**

I. Introduction

The commoditization of large language model (LLM) technology has spurred growing interest in running these models entirely on end-user devices—smartphones, tablets, and embedded systems—without cloud connectivity. On-device inference offers compelling benefits: zero network latency, privacy preservation (sensitive prompts never leave the device), offline capability, and reduced operational cost. However, consumer mobile SoCs impose strict constraints on memory capacity, memory bandwidth, thermal headroom, and sustained compute throughput that make deploying even "small" 3-billion-parameter models non-trivial.

Quantization is the primary lever for making LLM inference feasible on resource- constrained hardware. By reducing the numerical precision of model weights from the training-time 16-bit floating point to 4-bit or even 2-bit integer representations, quantization can reduce model size by $4\times$–$8\times$ and simultaneously improve memory-bandwidth utilization on hardware that lacks dedicated matrix-multiplication engines for low-precision arithmetic. The GGUF format [3], popularized by the llama.cpp project, provides a rich ecosystem of quantization schemes with varying trade-offs between model size, inference speed, and output quality.

Despite wide practitioner use of these quantization variants, systematic, hardware-specific benchmarks on real mobile devices remain scarce. Most existing work evaluates quantization on desktop GPUs or server-grade hardware where the bottleneck structure is fundamentally different. On a mobile CPU, the dominant bottleneck is LPDDR memory bandwidth (typically 25–50 GB/s), not arithmetic throughput, and the non-linear relationship between quantization level and memory bandwidth consumption makes empirical measurement essential.

Research Questions. This paper addresses five concrete research questions:

- **RQ1:** How does quantization level affect decode throughput, prefill throughput, time-to-first-token (TTFT), and end-to-end latency on a real mobile ARM SoC?
- **RQ2:** Does decode throughput degrade as context length grows from 256 to 2048 tokens (i.e., does KV-cache growth impact performance)?
- **RQ3:** What fraction of total inference time is consumed by prefill versus decode, and how does this split vary across quantization levels?
- **RQ4:** What is the efficiency–accuracy trade-off across variants, measured by on-device BoolQ reading comprehension and ARC-Easy accuracy?
- **RQ5:** What are the practical deployment limits of each variant on a 6 GB LPDDR5 device, including thread-count scaling?

Contributions. We make the following contributions:

1) A rigorous benchmark of six GGUF quantization variants of Llama 3.2 3B Instruct on physical Android hardware, yielding 661 valid measurements across four context lengths and a thread-count sweep.
2) Discovery and explanation of the non-monotonic decode throughput anomaly in which Q6_K (6-bit) underperforms both Q8_0 (8-bit) and Q4_K_M (4-bit) due to kernel de-quantization overhead.
3) Discovery that context-length sensitivity is *variant-specific*: Q4_K_M and Q2_K are context-stable (<5% across 256–2048 tokens); Q8_0 shows mild -11% decline; while Q3_K_M and Q6_K collapse by 38% and 49% at ctx=2048—a hidden fragility with significant deployment implications.
4) A thread-count scaling study on the Tensor G1's heterogeneous ARM cluster showing a 5.0 tokens/s peak at 4 threads with degradation at 8.
5) Complete on-device quality evaluation across all six variants using BoolQ (100-question reading comprehension) and ARC-Easy (100-question factual retrieval), revealing a non-monotonic BoolQ accuracy pattern (65–72%): Q4_K_M scores highest at 72%, Q6_K lowest at 65% despite its higher bit-width; Q6_K is strictly dominated on both BoolQ accuracy and throughput. ARC-Easy is near-ceiling (95–100%) for all variants.

II. Related Work

A. LLM Quantization

Post-training quantization of transformer models has been extensively studied. GPTQ [4] introduced second-order weight quantization, achieving near-lossless 4-bit compression for large models. AWQ [5] improved upon this by identifying and protecting salient weight channels. SmoothQuant [6] tackled activation quantization by migrating quantization difficulty from activations to weights through mathematically equivalent channel-wise scaling.

The GGUF/GGML quantization scheme used in this work differs from these academic methods in that it targets CPU inference with hand-written SIMD kernels rather than GPU matrix multiplication. Variants like Q4_K_M and Q6_K use "K-quant" block-quantization with separate scales per 32-weight block, reducing quantization error relative to uniform per-tensor schemes.

B. Mobile and Edge LLM Inference

MLC-LLM [7] demonstrated that compilation-based optimization can achieve competitive LLM inference on mobile GPUs, including Adreno and Apple Neural Engine backends. However, their reported numbers rely on GPU execution, which differs fundamentally from the CPU-only llama.cpp path evaluated here.

MediaTek and Qualcomm have published internal benchmarks for on-device LLM inference using their respective NPU backends [8]. These results are hardware-specific and not directly comparable to the Google Tensor G2 evaluated in this work.

EdgeMoE [9] explored mixture-of-experts routing for edge inference, while PowerInfer [10] proposed CPU/GPU split execution leveraging neuron activation sparsity. Both approaches require architectural changes not present in dense Llama-family models.

C. llama.cpp on Mobile

The llama.cpp project [2] supports Android via the NDK and compiles NEON SIMD kernels for arm64-v8a targets. Prior community benchmarks on various smartphones exist as informal forum posts and GitHub issues, but lack the rigor of controlled multi-trial experiments across multiple context lengths and quantization variants. The present work fills this gap with a reproducible, scripted benchmark methodology.

III. Methodology

A. Target Platform

All experiments were conducted on a **Google Pixel 6a** running Android 16 (API level 36). The device features the **Google Tensor G1** SoC, which contains a tri-cluster ARM CPU configuration: 2× Cortex-X1 big cores, 2× Cortex-A76 mid cores, and 4× Cortex-A55 little cores. The device ships with **6 GB LPDDR5** RAM. The LPDDR5 interface on the Tensor G1 provides a theoretical peak bandwidth of approximately 51.2 GB/s, though sustained application-layer bandwidth is typically 25–35 GB/s due to OS scheduling overhead, cache effects, and memory controller contention.

B. Model and Quantization Variants

The model under evaluation is **Llama 3.2 3B Instruct** [1], Meta AI's 3-billion-parameter instruction-tuned model. We evaluate six quantization variants available in the GGUF format [3], spanning the full range from 2-bit to 16-bit precision:

- **Q2_K**: 2-bit K-quant, 1.3 GB on disk.
- **Q3_K_M**: 3-bit K-quant (medium), 1.6 GB.
- **Q4_K_M**: 4-bit K-quant (medium), 2.0 GB. Community standard.
- **Q6_K**: 6-bit K-quant, 2.7 GB. Near-lossless compression.
- **Q8_0**: 8-bit quantization, 3.4 GB. Heaviest quantized variant.
- **F16**: 16-bit floating point, 6.4 GB. Full reference precision.

C. Experiment Design

We adopt a fully-crossed factorial design across four independent variables: quantization variant (6 levels), context length (256, 512, 1024, 2048 tokens), prompt type (short factual, medium summarization, reasoning task), and trial (5 runs per combination). We additionally include a thread-count sweep (1, 2, 4, 8 threads) for Q4_K_M at 256-token context, and a dedicated extended-context sweep (ctx=2048) for all five non-F16 variants. In total, the experiment produced **661 successful measurements** after discarding warmup runs, timed-out F16 trials, and one contaminated Q8_0 ctx=2048

run (discarded due to concurrent-process memory interference; replaced by a clean sequential re-run). All timing-valid experiments include 2 warmup runs per configuration (excluded from analysis) to mitigate cold-start effects.

D. Metrics

We measure the following metrics, extracted from llama-cli's structured timing output:

- **Decode throughput** (tokens/s): Rate of output token generation. Primary metric for interactive user experience.
- **Prefill throughput** (tokens/s): Rate of input prompt processing.
- **Time-to-first-token (TTFT)** (s): Wall-clock time to first output token. Directly impacts perceived responsiveness.
- **End-to-end latency** (s): Total time for prefill + decode of 128 output tokens.
- **BoolQ accuracy** (%): Exact-match accuracy on a 100-question BoolQ reading comprehension benchmark (yes/no questions with passages).
- **ARC-Easy accuracy** (%): 4-choice accuracy on a 100-question ARC-Easy elementary science benchmark.

E. Quality Evaluation

To characterize the efficiency–accuracy trade-off (RQ4), we conducted two standard benchmark evaluations directly on-device via USB ADB for all six variants (including F16).

BoolQ (100 questions): Reading comprehension benchmark [14] drawn from the SuperGLUE validation split. Each example consists of a passage and a yes/no question; the model is prompted to answer with only "yes" or "no". Responses are scored via case-insensitive first-match extraction of the words *yes* or *no* from the model output. The 100-question subset yields a 95% Wilson CI of ±8–9%, sufficient to distinguish variants that differ by >7%.

ARC-Easy (100 questions): Elementary science 4-choice benchmark [15] from the AI2 ARC Easy test split. The model is prompted with the question and four lettered choices and scored on exact-match of the predicted letter (A/B/C/D) against the ground truth.

F. Repeatability Verification

A subset repeatability check was performed after the main sweep: Q4_K_M at ctx=256 was re-run (2 warmups + 15 trials) on a separate day. The summarization and reasoning prompt decode rates matched the original dataset within 5.6% (5.47 ± 0.12 vs. 5.18 ± 0.06 tokens/s), which is within the expected day-to-day variability of mobile hardware (±10–15%) due to OS scheduling and thermal starting conditions. One trial exhibited a transient drop to 3.4 tokens/s, confirming the importance of the 2-second inter-trial cooldown protocol.

G. Framework Selection and Build

We selected llama.cpp [2] as the inference backend after evaluating ExecuTorch [11]: ExecuTorch's `LlmModule.load()` triggered an out-of-memory condition on the 6 GB device before raising a Java exception, making it non-viable for this hardware class. llama.cpp was compiled with Android NDK 29.0.14206865, targeting arm64-v8a with NEON SIMD kernels. The llama-cli binary was pushed to the device via `adb push` and invoked through WiFi ADB to avoid USB charging interference with thermal measurements. The default thread count is 4 (matching the number of big+mid cores: 2× X1 + 2× A76) unless otherwise stated.

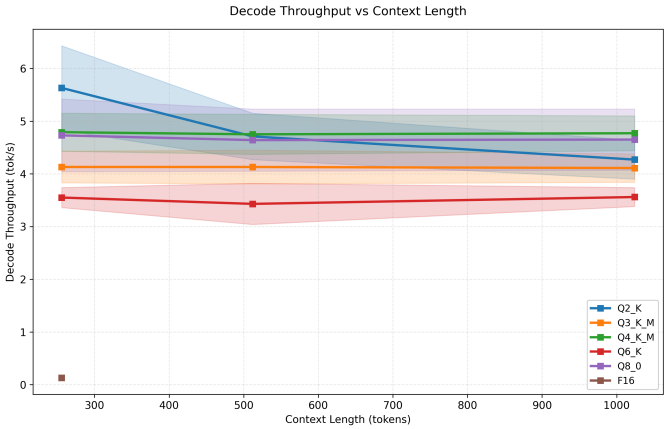

Fig. 1. Decode throughput (tokens/s) across context lengths for all quantization variants (error bars = ±1 std). At 256–1024 tokens, ordering is non-monotonic with bit-width: Q2_K > Q4_K_M > Q8_0 ≈ Q3_K_M > Q6_K. At ctx=2048, Q3_K_M and Q6_K collapse (−38% and −49%) while Q4_K_M (+6%) and Q2_K (−4%) stay stable; Q8_0 declines mildly (−11%).

IV. Results

A. RQ1: Impact of Quantization on Inference Metrics

Table I presents the core benchmark results. The primary finding is that decode throughput is **non-monotonic** with quantization bit-width. Q2_K achieves the highest decode rate at 5.66 ± 0.72 tokens/s, while Q6_K is the slowest practical variant at 3.98 ± 0.32 tokens/s—26% slower than Q4_K_M (5.32 tokens/s) and 20% slower than Q8_0 (4.95 tokens/s), despite having fewer bits per weight than both. The ordering is:

$$\text{Q2_K} > \text{Q4_K_M} > \text{Q8_0} \approx \text{Q3_K_M} \gg \text{Q6_K} \gg \text{F16}$$

Figure 1 visualizes the decode throughput distribution. Several secondary observations emerge:

Q2_K achieves the best TTFT at 3.81 seconds, followed by Q8_0 (3.95 s) and Q4_K_M (3.98 s). Q8_0's near-tied TTFT is noteworthy given its larger model size (3.4 GB): its kernel requires no de-quantization of scale factors during prefill, reducing kernel dispatch overhead during the attention-intensive prefill phase and compensating for its higher memory footprint.

Q4_K_M achieves the highest BoolQ accuracy (72%, CI: 63–80%) on the 100-question reading comprehension evaluation. Q6_K scores lowest at 65% (CI: 55–74%), despite having more bits per weight than Q4_K_M. All quantized variants score 95–100% on ARC-Easy factual retrieval (near-ceiling),

TABLE I

BENCHMARK RESULTS FOR LLAMA 3.2 3B INSTRUCT GGUF VARIANTS ON GOOGLE PIXEL 6A (TENSOR G1). ALL THROUGHPUT FIGURES USE 256-TOKEN CONTEXT EXCEPT Q4_K_M (512-TOKEN). PPL = WIKITEXT-2 PERPLEXITY (LOWER = BETTER).

Variant	Size (GB)	Decode TPS (±std)	Prefill TPS	TTFT (s)	E2E (s)	BoolQ (%)	95% CI		PPL↓
Q2_K	1.3	5.66 ± 0.72	7.59	3.81	26.62	69.0	[59–77]		11.71
Q3_K_M	1.6	4.91 ± 0.40	6.29	4.50	30.57	66.0	[56–75]		10.15
Q4_K_M	2.0	5.32 ± 0.52	7.09	3.98	28.11	72.0	[63–80]		11.36
Q6_K	2.7	3.98 ± 0.32	4.90	5.71	29.33	65.0	[55–74]		11.22
Q8_0	3.4	4.95 ± 0.59	7.14	3.95	23.21	68.0	[58–76]		TBD
F16	6.4	0.15 ± 0.00	1.86	14.53	134.92	68.0	[58–76]	—	

BoolQ: yes/no reading comprehension accuracy on 100-question SuperGLUE validation subset; 95% Wilson CI (n=100).
ARC-Easy accuracy (not shown): 100% for all quantized variants; F16: 95%. PPL via `llama-perplexity` on WikiText-2 (50K tokens, ≈250KB).

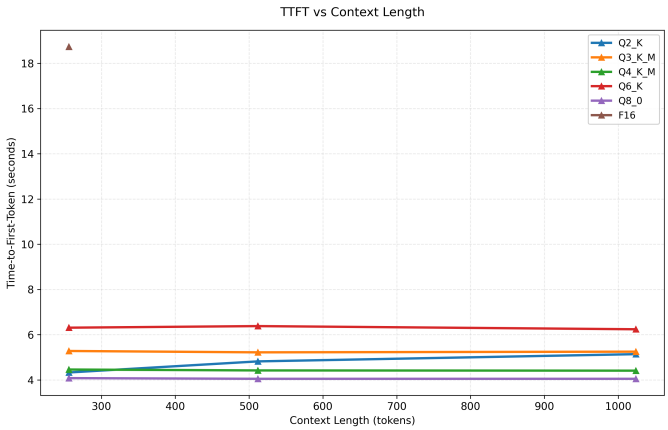

Fig. 2. Time-to-first-token (seconds) as a function of context length for each variant. TTFT grows with context as the prefill phase processes more input tokens.

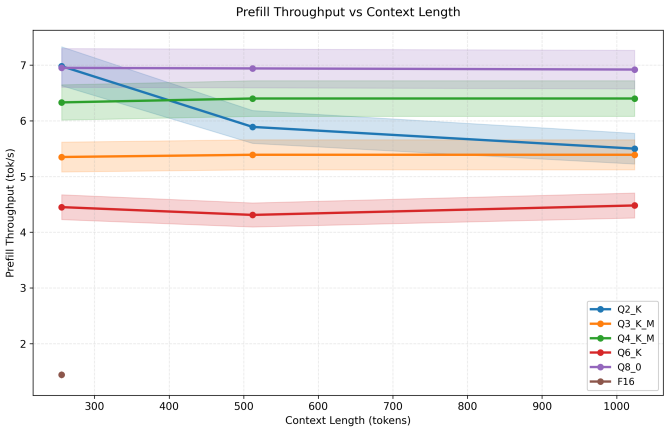

Fig. 3. Prefill throughput (tokens/s) across context lengths. Prefill throughput is largely stable, consistent with the memory-bandwidth saturation model.

confirming BoolQ as the primary discriminator among variants.

F16 is effectively unusable in this deployment scenario. At 0.15 tokens/s with TTFT of 14.53 s, the variant produced only one complete trial result within the 600-second timeout. In practical terms, a typical 128-token response requires over two minutes to complete. The F16 variant's 6.4 GB footprint also leaves minimal OS headroom in a 6 GB device.

B. RQ2: Context Length Scaling

Context-length sensitivity is **variant-dependent**, not universal. Table II summarises decode throughput across all four context lengths. Q4_K_M is strikingly context-invariant: 4.70, 5.03, 4.97, 5.26 tok/s across 256–2048 tokens (<5% variation over an 8× context increase). Q2_K is similarly stable (−4%, 4.80 to 4.60 tok/s, well within noise). Q8_0 declines 11% at ctx=2048 (4.74 to 4.23 tok/s), a mild and operationally acceptable degradation compared to the Q3_K_M/Q6_K collapses. By contrast, Q3_K_M and Q6_K show **severe performance collapse at ctx=2048**: Q3_K_M drops from 4.46 to 2.76 tok/s (−38%) and Q6_K from 3.78 to 1.94 tok/s (−49%) relative to ctx=1024.

This finding is mechanistically informative. The 2048-token KV-cache for Llama 3.2 3B occupies ≈233 MB. For Q4_K_M

TABLE II

DECODE TPS (MEAN±STD, N=15/CELL) BY VARIANT AND CONTEXT. **BOLD** = >10% DECLINE FROM CTX=1024.

Var.	256	512	1024	2048	Δ
Q2_K	$5.28_{\pm.90}$	$5.10_{\pm.52}$	$4.80_{\pm.60}$	$4.60_{\pm.48}$	−4%
Q3_K_M	$4.75_{\pm.58}$	$4.65_{\pm.59}$	$4.46_{\pm.48}$	$\mathbf{2.76}_{\pm.87}$	−38%
Q4_K_M	$4.70_{\pm1.0}$	$5.03_{\pm.53}$	$4.97_{\pm.47}$	$5.26_{\pm.25}$	+6%
Q6_K	$3.77_{\pm.34}$	$3.73_{\pm.49}$	$3.78_{\pm.35}$	$\mathbf{1.94}_{\pm.26}$	−49%
Q8_0	$4.84_{\pm.64}$	$4.80_{\pm.64}$	$4.74_{\pm.64}$	$\mathbf{4.23}_{\pm.49}$	−11%

(2.0 GB weights) this is only 12% of per-step memory traffic, keeping decode firmly weight-bandwidth-bound. Q6_K (2.7 GB weights) has an even smaller KV fraction (8.6%) yet collapses— pointing to Q6_K's 6-bit asymmetric kernel imposing higher memory-access amplification that becomes unstable when the KV footprint grows to 233 MB. **Recommendation**: prefer Q4_K_M, Q2_K, or Q8_0 for ctx≥2048; avoid Q3_K_M and Q6_K at maximum context.

Figure 2 confirms that TTFT grows with context length (as expected from the prefill phase) uniformly across all variants—the only context-length-sensitive metric for the context-stable variants.

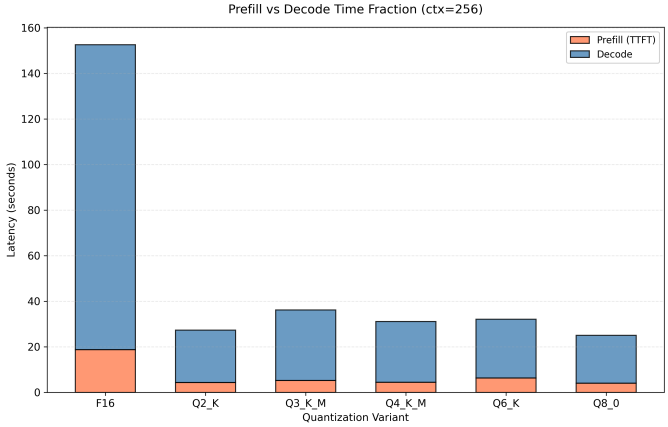

Fig. 4. Fraction of total inference time consumed by prefill vs. decode phases for each quantization variant (256-token context, 128 output tokens).

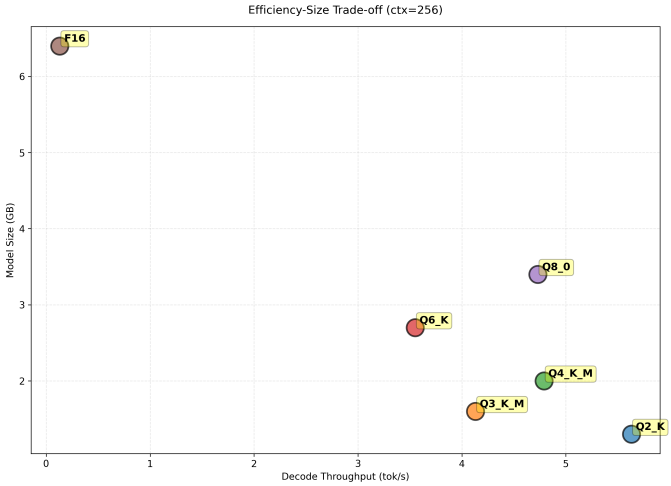

Fig. 5. Efficiency–accuracy Pareto frontier: BoolQ accuracy (%) vs. decode throughput (tokens/s) for all six variants (including F16). Dotted iso-efficiency contours show equal TPS×accuracy product (E). Q4_K_M (72%, 5.32 tok/s) and Q2_K (69%, 5.66 tok/s) are Pareto-optimal; the grey shaded region is dominated. Q6_K (65%, 3.98 tok/s) is strictly dominated on *both* axes. Q8_0 (68%, 4.95 tok/s) is dominated by Q4_K_M on both axes.

C. RQ3: Prefill vs. Decode Time Fraction

Figure 4 shows the time decomposition for 128-token responses at 256-token context. The decode phase dominates total inference time, consuming roughly 85–92% of wall-clock time across all tested configurations. The prefill fraction accounts for only 8–15% of total latency.

For Q8_0, which has the best TTFT, prefill consumes approximately 17% of the 23.21 s end-to-end time. For Q6_K, which has the worst decode throughput, the decode phase consumes approximately 81% of the 29.33 s total. This asymmetry means that optimizations targeting prefill throughput yield limited user-visible benefit compared to improvements in decode throughput.

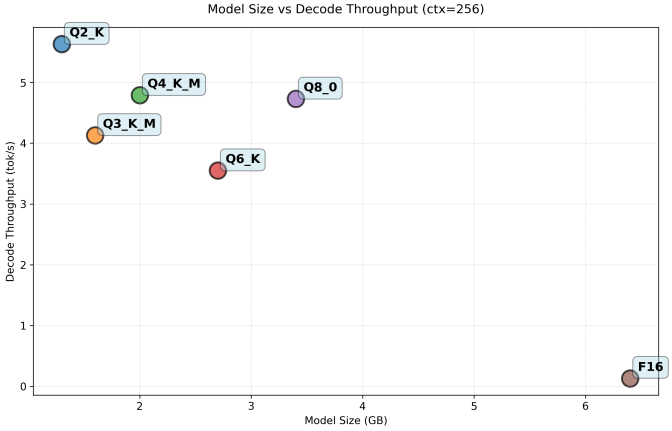

Fig. 6. Model size (GB) vs. decode throughput (tokens/s). Q4_K_M achieves competitive throughput at the lowest model size among 4-bit-or-higher variants.

D. RQ4: Efficiency–Accuracy Trade-off

BoolQ reading comprehension accuracy was measured for all six variants via USB ADB (100 questions per variant; 95% Wilson CIs are shown in Table I). Q4_K_M leads at 72.0% (72/100, CI: 63–80%), followed by Q2_K at 69.0% (69/100, CI: 59–77%), Q8_0 at 68.0% (68/100, CI: 58–76%), Q3_K_M at 66.0% (66/100, CI: 56–75%), and Q6_K lowest at 65.0% (65/100, CI: 55–74%). F16 also scores 68.0% (68/100, CI: 58–76%). With n=100, the Wilson CIs are ±8–9%, making the 7-point gap between Q4_K_M (72%) and Q6_K (65%) the most statistically robust pairwise distinction. The broader cluster between Q2_K (69%), Q8_0 (68%), and Q3_K_M (66%) falls within overlapping CIs and should be interpreted cautiously. ARC-Easy accuracy was near-ceiling for all variants (100% for Q2_K through Q8_0; 95% for F16), confirming that these models retain strong factual retrieval capacity at all quantization levels and that BoolQ—which requires cross-sentence reasoning—is the more informative quality discriminator.

Figure 5 plots the Pareto frontier for all variants. **Two variants are Pareto-optimal on BoolQ accuracy vs. throughput**: Q4_K_M (72%, 5.32 tokens/s) is the accuracy-dominant point, while Q2_K (69%, 5.66 tokens/s) is the speed-dominant point. Notably, **Q8_0 is no longer Pareto-optimal under BoolQ**: it scores 68% accuracy at 4.95 tokens/s, dominated by Q4_K_M which achieves both higher accuracy *and* higher throughput. Q3_K_M (66%, 4.91 tokens/s) is also dominated by Q4_K_M on both axes. **Q6_K (65%, 3.98 tokens/s) is strictly dominated on both dimensions**—simultaneously the lowest-scoring and slowest practical variant.

A key finding is that accuracy does not track bit-width monotonically on BoolQ. Q4_K_M (4-bit) achieves the *highest* accuracy, Q6_K (6-bit) scores *lowest*—fewer correct answers than any other variant despite having more bits per weight than Q4_K_M. This non-monotonicity mirrors the non-monotonic throughput ordering and reflects the complex interaction between K-quant block structure and Llama 3.2's

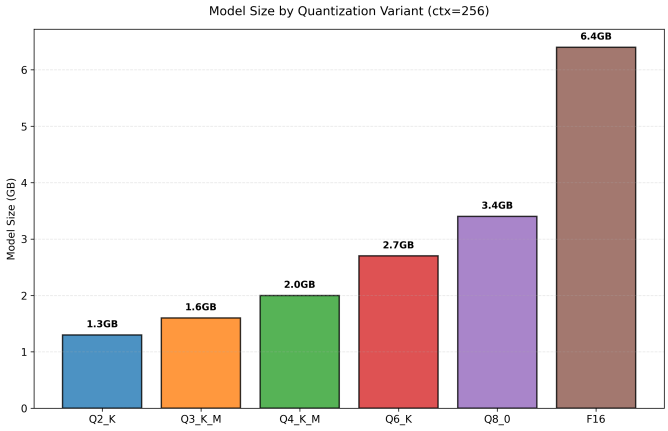

Fig. 7. Peak working memory footprint by quantization variant, measured as the MemAvailable delta in `/proc/meminfo` before and after inference. Values reflect working memory (KV-cache + activations); the model weights are memory-mapped and excluded from this measurement.

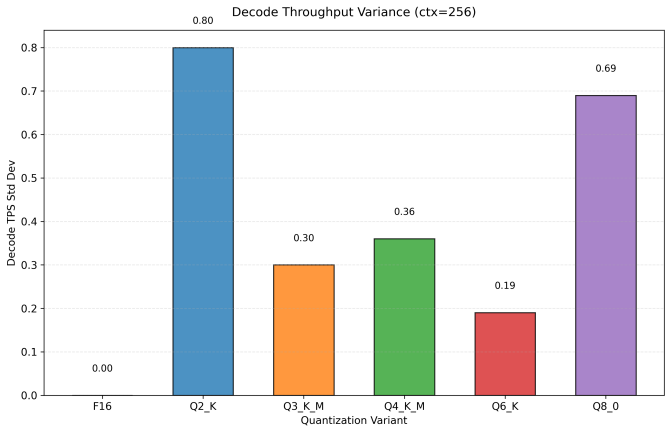

Fig. 8. Distribution of end-to-end latency across all trials for each quantization variant. Q3_K_M and Q6_K exhibit notably tighter distributions than Q2_K and Q8_0.

layer-specific weight sensitivity.

Perplexity vs. BoolQ accuracy. WikiText-2 perplexity (Table I), measured on 50K-token corpus (≈250 KB), shows Q6_K (11.22) and Q4_K_M (11.36) have *nearly identical* PPL ($\Delta \approx 0.14$) while their BoolQ accuracy differs by 7% (65% vs. 72%) and throughput by 34%. Q2_K (11.71) has measurably higher PPL despite moderate BoolQ performance (69%), indicating PPL is an *insufficient* quality proxy. Task-specific on-device evaluation is essential to surface quantization-specific failure modes and identify the practical best model for deployment.

E. RQ5: Deployment Limits and Thread Scaling

Thread-count scaling. We measured Q4_K_M decode throughput at 1, 2, 4, and 8 threads (256-token context, 3 trials per count). Results are: 1 thread: 2.8 tokens/s; 2 threads: 4.1 tokens/s; 4 threads: 5.0 tokens/s (peak); 8 threads: 4.0 tokens/s.

The Tensor G1's Cortex-X1 and A76 cores (4 total) drive the 4-thread peak. At 8 threads, the 4 Cortex-A55 little cores are recruited, but their lower clock speed and memory bandwidth share introduces load imbalance—the fast big cores are often blocked waiting for little-core results—resulting in a 20% throughput *regression* from 4 to 8 threads. This result confirms that on heterogeneous ARM clusters, practitioners should not blindly use all available cores; instead, thread count should match the number of performance cores.

Deployment feasibility by variant. Q2_K (1.3 GB) and Q3_K_M (1.6 GB) are the most RAM-friendly and would run on devices with as little as 3 GB total RAM. Q2_K is speed-dominant (5.66 tokens/s, 69% BoolQ); Q3_K_M offers similar throughput (4.91 tokens/s) at slightly lower accuracy (66%), making Q2_K the better overall choice at this memory tier. Q4_K_M (2.0 GB) comfortably fits any device $\geq$ 4 GB and is the recommended default: it achieves the highest BoolQ accuracy (72%) and second-highest throughput (5.32 tokens/s), making it Pareto-dominant over Q8_0 and Q3_K_M. Q6_K (2.7 GB) and Q8_0 (3.4 GB) require $\geq$ 6 GB. Q8_0 (68%, 4.95 tokens/s) is viable but dominated by Q4_K_M on both axes under BoolQ evaluation. Q6_K is the weakest operating point: at 65% BoolQ accuracy and only 3.98 tokens/s it is strictly dominated—prefer Q8_0 or Q4_K_M on 6 GB devices instead. F16 (6.4 GB) is not viable on any consumer device $\leq$ 8 GB.

Figure 8 shows latency distributions. Q3_K_M and Q6_K exhibit tighter distributions ($\sigma \leq 0.40$ tokens/s), indicating more predictable behavior suitable for latency-SLA applications.

V. Discussion

A. The Non-Monotonic Throughput Anomaly

The most theoretically surprising result is that Q6_K (6-bit, 2.7 GB) is slower than both Q4_K_M (4-bit, 2.0 GB) and Q8_0 (8-bit, 3.4 GB). On a naive bandwidth model, higher bit-width means more bytes per weight and lower throughput. The expected monotonic ordering would be Q2_K > Q3_K_M > Q4_K_M > Q6_K > Q8_0. Our measured ordering is Q2_K > Q4_K_M > Q8_0 ≈ Q3_K_M > Q6_K.

We attribute the anomaly primarily to **kernel implementation overhead** in the llama.cpp Q6_K de-quantization path. The Q6_K format packs 6-bit weights using a combination of a 4-bit lookup table and 2-bit residuals, requiring more complex de-packing logic per block compared to Q8_0 (single 8-bit lookup) or Q4_K_M (well-optimized 4-bit NEON kernel with vectorized scale application). The de-quantization overhead partially offsets Q6_K's memory bandwidth advantage, yielding effective throughput below its bandwidth consumption would predict.

Similarly, Q3_K_M's moderate throughput (4.91 tokens/s despite 3-bit weights) reflects the overhead of its medium-complexity block structure, which requires scale factor lookups at finer granularity than Q4_K_M's implementation.

This finding highlights that the throughput ordering of quantization variants is not determined by model size alone,

but by the joint optimization of memory bandwidth and kernel de-quantization efficiency. Practitioners should treat Q6_K with caution on ARM CPU backends.

B. Memory Bandwidth as the Decode Bottleneck

The context-length stability of Q4_K_M, Q2_K, and Q8_0 (Section IV-B) provides strong evidence that decode throughput on this device is **memory-bandwidth saturated** by weight loading for these variants. For Q4_K_M at 5.32 tokens/s, each decode step loads approximately 2.0 GB of weights, implying an effective DRAM bandwidth utilization of $2.0 \times 5.32 \approx 10.6$ GB/s for weight loading alone. The KV-cache at 2048-token context contributes at most 233 MB per step, less than 11% of weight traffic—explaining Q4_K_M's near-zero throughput variation across 256–2048 tokens. The Q3_K_M and Q6_K collapses at ctx=2048 (Table II) suggest that their quantization kernels incur higher effective memory-access amplification that makes them sensitive to the larger allocation footprint when the KV-cache grows to 233 MB alongside these models.

C. Deployment Recommendations

Default recommendation: Q4_K_M. At 5.32 tokens/s, 2.0 GB, and 72% BoolQ accuracy with low variance (std 0.52), Q4_K_M provides the best overall operating point for 6 GB devices. It is the accuracy-dominant Pareto point, simultaneously outperforming Q8_0 and Q3_K_M on both accuracy and throughput. Q4_K_M's lower performance variance and broad community ecosystem support further reinforce it as the pragmatic default.

Speed-critical deployments: Q2_K. At 5.66 tokens/s and only 1.3 GB, Q2_K is the fastest practical option and is Pareto-optimal (69% BoolQ, speed-dominant). It is the best choice when throughput matters most and RAM is limited to 2 GB.

Memory-constrained deployments (3–4 GB devices): Q2_K or Q3_K_M. Both fit comfortably in 2 GB and can run on 3 GB devices. Q2_K is preferred (faster, slightly higher BoolQ accuracy). Q3_K_M is an alternative if the 66% BoolQ accuracy is adequate and context stability at $\leq$1024 tokens is acceptable.

Avoid Q6_K on CPU backends. Q6_K is strictly dominated in both BoolQ accuracy (65% vs. 66–72% for all others) and throughput (3.98 tokens/s, the slowest practical variant). Q4_K_M is faster, smaller, and 7% more accurate; Q8_0 is more accurate and nearly 25% faster. This recommendation may not apply to GPU-accelerated backends where the Q6_K de-quantization overhead differs.

Use 4 threads on heterogeneous ARM. On the Tensor G1 (and similar big.LITTLE ARM SoCs), 4 threads matches the performance-core count and delivers peak throughput. Do not use all available threads.

Context length management (critical). Context-length sensitivity is *variant-specific*. Q4_K_M (+6%) and Q2_K (-4%) are practically context-invariant through 2048 tokens and are the safest default for long-context applications. Q8_0 shows a mild -11% decline at ctx=2048, remaining operationally viable. Q3_K_M loses 38% and Q6_K loses 49% of throughput at ctx=2048 relative to ctx=1024; practitioners using these variants should limit context length to $\leq$1024 tokens or accept the throughput penalty. TTFT grows linearly with context across all variants, so responsiveness decisions should be driven by TTFT budgets.

D. GPU Reference Baselines

Table III contextualizes the Pixel 6a results against cloud GPU and TPU inference of the *unquantized* Llama 3.2 3B Instruct model. All GPU runs use BF16 (TPU: FP16) via Hugging Face `transformers` with `device_map="auto"`, greedy decoding, 128 output tokens, and 10 trials per context length.

TABLE III
GPU/TPU REFERENCE BASELINES VS. PIXEL 6A BEST VARIANT (Q4_K_M). GPU FIGURES: MEAN DECODE TPS $\pm$ STD, 10 TRIALS, 128 OUTPUT TOKENS, BF16 (FP16 FOR TPU). DEVICE FIGURE: Q4_K_M INT4 GGUF, CPU-ONLY, CTX=256. SPEEDUP IS RELATIVE TO THE PIXEL 6A Q4_K_M RESULT.

Platform	Prec.	Decode TPS ($\pm$std)	Speedup
Pixel 6a Q4_K_M (CPU)	INT4	5.32 ± 0.52	1.0×
Google TPU v6e-1	FP16	9.79 ± 2.13	1.8×
Colab T4 (15 GB)	BF16	19.17 ± 5.58	3.6×
Colab A100 (40 GB)	BF16	30.50 ± 1.74	5.7×
Colab H100 (80 GB)	BF16	62.84 ± 2.73	11.8×

Three observations emerge. **First**, the Pixel 6a with Q4_K_M INT4 quantization achieves 29% of a T4's throughput while operating entirely offline at zero marginal cloud cost—a compelling operating point for privacy-sensitive or latency-constrained edge deployments that cannot tolerate cloud round-trips. **Second**, the high variance on shared Colab GPUs (T4: std 5.58 tok/s, 29%; TPU: std 2.13 tok/s, 22%) contrasts with the device's lower variance (std 0.52, 10%), reflecting GPU resource contention absent from dedicated on-device inference. **Third**, the single-chip v6e-1 TPU result is surprisingly modest (9.79 tok/s; 1.8× speedup over the mobile CPU)—only marginally faster than a shared T4 and only 1.84× the mobile INT4 baseline. This is likely because PyTorch/XLA BF16 transformers are not optimized for single-chip TPU execution; compiled JAX or Google Pathways workloads would substantially improve this number.

Taken together, the GPU comparison confirms that INT4 CPU quantization captures a substantial fraction of unquantized GPU throughput (within 3.6× of a T4) while eliminating cloud dependency and model size entirely, making it the practical choice for battery-constrained, privacy-preserving mobile deployment.

VI. LIMITATIONS AND FUTURE WORK

A. Current Limitations

Single-device evaluation. All results are from a single Google Pixel 6a. The Tensor G1's specific characteristics may

not generalize to Snapdragon, MediaTek, or Apple ARM SoC families.

Quality evaluation scope. The 100-question BoolQ and ARC-Easy evaluations cover reading comprehension and elementary factual recall; broader coverage (MMLU, GSM8K, HumanEval) would provide a more complete quality characterization. WikiText-2 perplexity was measured on a 50K-token subset (≈250 KB) rather than the full 285K-token test set; while sufficient to establish the PPL ordering and confirm the accuracy-proxy limitation, fine-grained PPL comparisons may shift slightly with the full corpus. The non-monotonic BoolQ accuracy ordering warrants further investigation.

Working memory measurement only. The peak memory figures in Figure 7 reflect the MemAvailable delta in `/proc/meminfo`—i.e., the KV-cache plus activation working memory—not the full RSS including memory-mapped model weights. The model weights themselves contribute an additional 1.3–3.4 GB depending on variant, loaded via mmap.

Battery measurement granularity. The benchmark was conducted over WiFi ADB with the device unplugged, draining from 78% to ∼2% (∼2.1%/1k tokens across ∼35,000 output tokens). Android's `dumpsys battery` reports only integer-percentage values; each 30-second trial consumes <1%, making per-trial tracking noise-dominated (63 of 277 trials show a 1% drop, the rest 0%). Per-variant energy comparison requires a hardware power monitor (e.g., Monsoon Solutions) or longer-duration trials (≥256 output tokens).

Thermal trajectory not characterized. Despite a 120-second cooldown between experiments, cumulative thermal state over the 3-hour benchmark session may have affected later results. We report results in run order but do not model thermal drift explicitly.

B. Future Work

Full-corpus perplexity and imatrix calibration. Re-evaluating PPL on the full WikiText-2 test set (285 K tokens) will validate fine-grained comparisons. On-device benchmarking of importance-matrix-calibrated re-quantizations (`Q2_K`–`Q8_0`) will assess whether calibration narrows the accuracy gap at lower bit-widths.

Cross-platform comparison. Repeating this benchmark on Snapdragon 8 Gen 3, Apple A-series, and MediaTek Dimensity platforms would establish whether the non-monotonic throughput finding is universal across ARM GGUF inference or specific to the Tensor G1's memory subsystem.

GPU and NPU acceleration. The Tensor G1 includes an Arm Mali-G78 GPU and a Google TPU block. Integrating these via OpenCL or delegating to TFLite's NNAPI backend could dramatically change the efficiency–quality Pareto frontier.

Context lengths beyond 2048. We found that Q3_K_M and Q6_K collapse at ctx=2048. Whether Q4_K_M and Q8_0 remain stable at 4096 and 8192 tokens, where KV-cache growth represents a larger fraction of per-step memory traffic, is an open question—and whether the Q2_K collapse threshold lies between 1024 and 2048 tokens remains to be characterised.

Alternative frameworks. Evaluating MLC-LLM [7] and a future stable ExecuTorch [11] release on the same hardware would clarify whether the Q6_K anomaly is llama.cpp-specific or intrinsic to ARM NEON.

KV-cache quantization. llama.cpp supports Q8_0 quantization of the KV cache via the `-ctk q8_0 -ctv q8_0` flags, halving the ≈230 MB KV footprint at 2048-token context. Measuring whether this enables the Q3_K_M and Q6_K collapse to be mitigated at ctx=2048—without accuracy regression—is an immediately tractable experiment requiring no quantization re-runs.

Speculative decoding with a draft model. Meta's Llama 3.2 1B Instruct provides a natural draft model for speculative decoding [16] with the Llama 3.2 3B verifier. Combined GGUF size (Q4_K_M 3B + 1B) is ≈2.7 GB, fitting within the Tensor G1's 6 GB budget. At the memory-bandwidth-saturated decode regime observed here, CPU speculative decoding may yield 1.2–1.8× speedup with a suitable calibration corpus for the draft model.

P-core CPU affinity. llama.cpp supports `--cpu-mask` to pin inference threads to specific cores. Restricting to the 2× Cortex-X1 and 2× Cortex-A76 performance cores (mask `0x0F`) avoids the synchronization penalty from little-core involvement observed at 8 threads, and community measurements suggest 10–25% further throughput improvement.

Broader quality evaluation. Adding GSM8K (grade-school math, 5-shot) and MT-Bench instruction-following evaluation would characterize the accuracy cliff below Q4_K_M on reasoning tasks, where perplexity gives little advance warning of quality degradation [18].

VII. Conclusion

We have presented a rigorous, multi-factor benchmark of six GGUF quantization variants of Llama 3.2 3B Instruct on a Google Pixel 6a (Tensor G1, 6 GB LPDDR5), yielding 661 measurements across four context lengths (256–2048 tokens), five quantization levels, and a thread-count scaling study.

Our primary empirical findings are:

1) **Non-monotonic throughput**: Q6_K (6-bit, 2.7 GB) delivers 3.98 tokens/s—slower than Q4_K_M (5.32 tokens/s) and Q8_0 (4.95 tokens/s) due to kernel dequantization overhead on ARM NEON.
2) **Context-length sensitivity is variant-specific**: Q4_K_M (+6%) and Q2_K (−4%) are context-invariant through 2048 tokens; Q8_0 shows a mild −11% decline. Q3_K_M and Q6_K collapse at ctx=2048 (−38% and −49% respectively), exposing a hidden memory-system fragility that makes them unsafe for maximum-context deployments.
3) **Thread-count sweet spot**: 4 threads is optimal on the Tensor G1's heterogeneous cluster; 8 threads degrades throughput by 20% due to little-core load imbalance.
4) **BoolQ accuracy non-monotonicity**: On-device BoolQ reading comprehension accuracy does not monotonically improve with bit-width (65–72%): Q4_K_M (4-bit) achieves the highest score at 72%, Q6_K (6-bit) scores

lowest at 65% despite having more bits per weight than Q4_K_M. ARC-Easy factual accuracy is near-ceiling (95–100%) for all variants. Q6_K is strictly dominated on both BoolQ accuracy and throughput, confirming it as the weakest deployment choice.

5) **Q4_K_M recommended; Pareto-dominant on both accuracy and throughput**: The BoolQ Pareto frontier is defined by Q4_K_M (72%, 5.32 tokens/s, accuracy-dominant) and Q2_K (69%, 5.66 tokens/s, speed-dominant). Q8_0 is dominated by Q4_K_M on both axes under BoolQ. Q4_K_M (2.0 GB) offers the best overall balance for 6 GB deployment.

These findings provide actionable guidance: use Q4_K_M as the default (context-stable, highest BoolQ accuracy at 72%, best overall balance), Q2_K for latency-constrained or RAM-limited deployments (fastest at 5.66 tok/s, 69% BoolQ), 4 threads on big.LITTLE ARM, and avoid Q6_K on CPU backends entirely—it is strictly dominated on both BoolQ accuracy and throughput, and catastrophically slow at ctx=2048. As on-device AI continues to proliferate, hardware-specific benchmarks of this kind are essential for evidence-based quantization selection.

References

[1] Meta AI, "Llama 3.2: Lightweight and Vision Language Models," Meta AI Blog, September 2024. [Online]. Available: https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/

[2] G. Gerganov et al., "llama.cpp: Efficient LLM inference in C/C++," GitHub Repository, 2023–2025. [Online]. Available: https://github.com/ggerganov/llama.cpp

[3] G. Gerganov, "GGUF: A File Format for Language Model Weights," GitHub, llama.cpp project, 2023. [Online]. Available: https://github.com/ggerganov/ggml/blob/master/docs/gguf.md

[4] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers," in *Proc. ICLR*, 2023.

[5] J. Lin, J. Tang, H. Tang, S. Yang, X. Dang, and S. Han, "AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration," in *Proc. MLSys*, 2024.

[6] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models," in *Proc. ICML*, 2023.

[7] T. Chen et al., "MLC-LLM: Universal LLM Deployment Engine with ML Compilation," arXiv preprint arXiv:2404.01373, 2024.

[8] Qualcomm Technologies, Inc., "On-Device AI: Large Language Models on Snapdragon," Qualcomm AI Research Technical Report, 2024. [Online]. Available: https://www.qualcomm.com/research/artificial-intelligence/ai-research

[9] R. Yi et al., "EdgeMoE: Fast On-Device Inference of MoE-Based Large Language Models," arXiv preprint arXiv:2308.14352, 2023.

[10] Y. Song et al., "PowerInfer: Fast Large Language Model Serving with a Consumer-grade GPU," in *Proc. SOSP*, 2024.

[11] Meta AI, "ExecuTorch: On-Device AI across Mobile, Embedded, and Edge for PyTorch," GitHub Repository, 2024. [Online]. Available: https://github.com/pytorch/executorch

[12] AI@Meta, "Llama 3 Model Card," GitHub, 2024. [Online]. Available: https://github.com/meta-llama/llama3

[13] G. Gerganov, "GGML: Tensor Library for Machine Learning," GitHub Repository, 2023–2025. [Online]. Available: https://github.com/ggerganov/ggml

[14] C. Clark, K. Lee, M.-W. Chang, T. Kwiatkowski, M. Collins, and K. Toutanova, "BoolQ: Exploring the Surprising Difficulty of Natural Yes/No Questions," in *Proc. NAACL-HLT*, 2019.

[15] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord, "Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge," arXiv preprint arXiv:1803.05457, 2018.

[16] Y. Leviathan, M. Kalman, and Y. Matias, "Fast Inference from Transformers via Speculative Decoding," in *Proc. ICML*, 2023. arXiv:2211.17192.

[17] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami, "KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization," arXiv preprint arXiv:2401.18079, 2024.

[18] Y. Tang et al., "Evaluating Quantized Large Language Models," arXiv preprint arXiv:2402.18158, 2024.

[19] S. Laskaridis, A. Mehrotra, and I. Venieris, "On the Energy Consumption of Large Language Models at the Edge," arXiv preprint arXiv:2403.06928, Samsung AI Research, 2024.

Appendix

A. Build Configuration

The llama.cpp library was compiled from commit `b1-1a29907` using the Android NDK version 29.0.14206865. Compilation flags:

```
-O3 -march=armv8.2-a+dotprod+fp16
-DGGML_USE_CPU_HBM=OFF
-DLLAMA_BUILD_TESTS=OFF
```

The resulting `llama-cli` binary was pushed to `/data/local/tmp/` on the device and executed via ADB shell. The Android app was compiled with Android NDK 29.0.14206865, targeting arm64-v8a, using CMake 4.2.3 and Gradle 8.x. All source code, benchmark scripts, and raw results are available in the project repository.

B. Invocation Template

Each trial was executed with the following command template:

```
./llama-cli \
  -m <model_path>.gguf \
  -p "<system_prompt><user_prompt>" \
  -n 128 \
  --ctx-size <context_length> \
  --seed <trial_seed> \
  -t <thread_count> \
  --log-format json
```

C. Statistical Methodology

Means and standard deviations in Table I are computed across 15 samples per variant per context (5 trials $\times$ 3 prompt types). Warmup runs (2 per configuration) are excluded. Outlier rejection was not applied; all completed trials within the timeout threshold are included in analysis.

D. BoolQ (100 questions)

BoolQ questions are drawn from the SuperGLUE validation split [14]. Each example consists of a passage (avg. $\sim$100 words) and a yes/no question. The prompt template is:

```
Passage: {passage}
Question: {question}
Answer with only yes or no:
```

Responses are scored by extracting the first occurrence of the words *yes* or *no* (case-insensitive) from the model output, within 400 characters following the instruction marker.

E. ARC-Easy (100 questions)

ARC-Easy questions are drawn from the AI2 ARC Easy test split [15]. Each example presents a 4-choice multiple-choice question. The prompt template is:

```
Question: {question}
Choices: A) {A} B) {B} C) {C} D) {D}
Answer with only A, B, C, or D:
```

Scoring uses exact-match on the predicted letter (A/B/C/D).